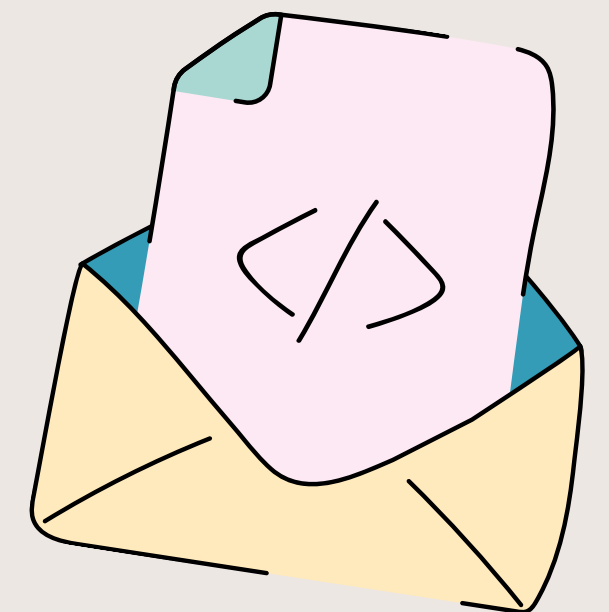
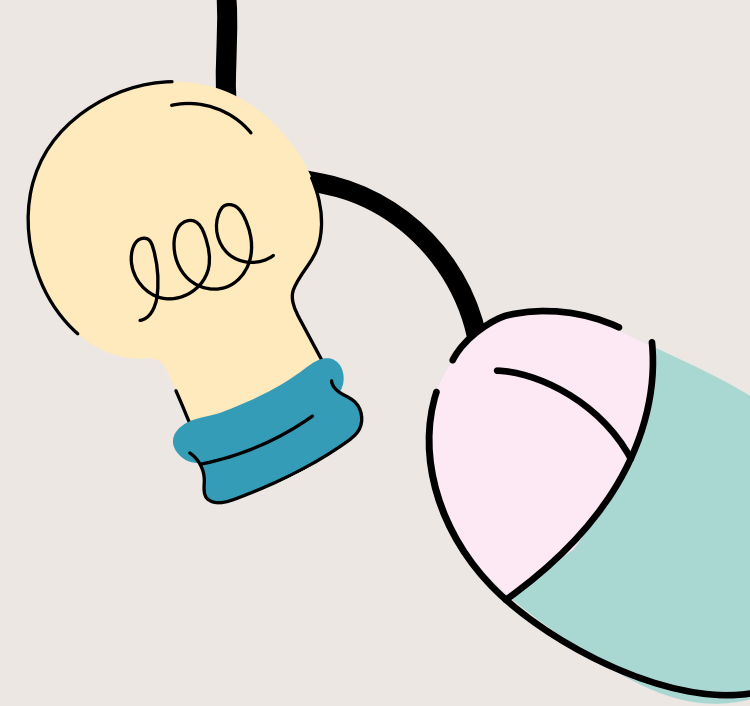
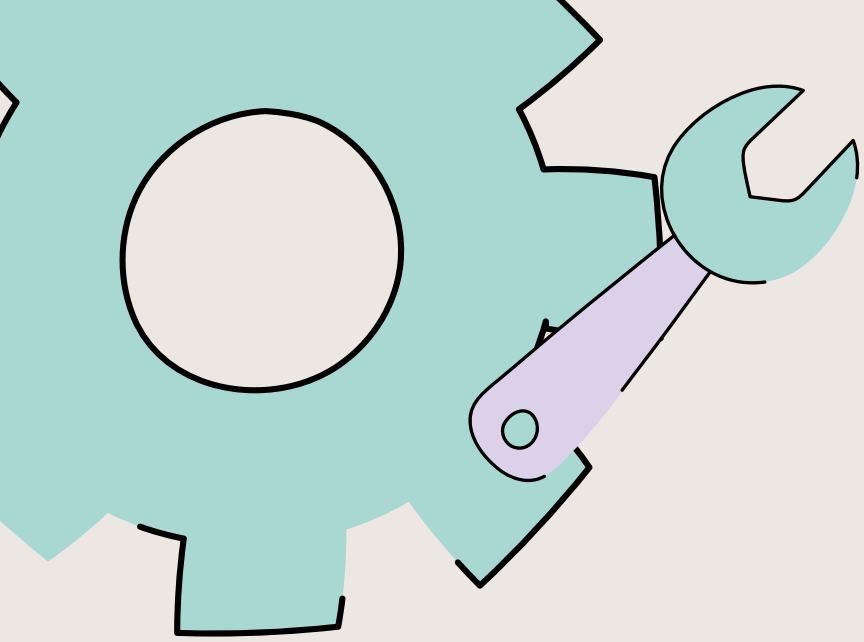
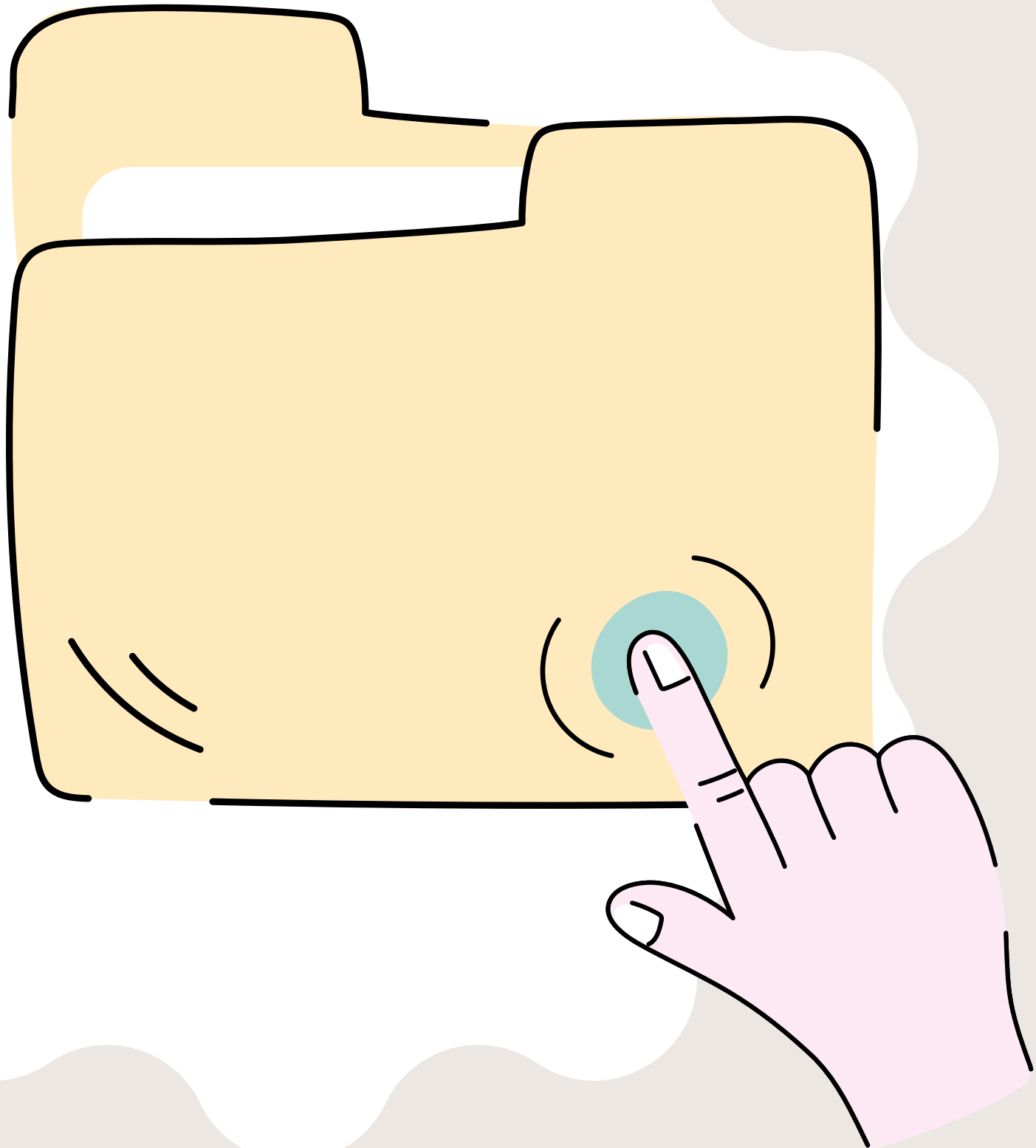


Nº 14: Clases y Programación Orientada a Objetos (POO)



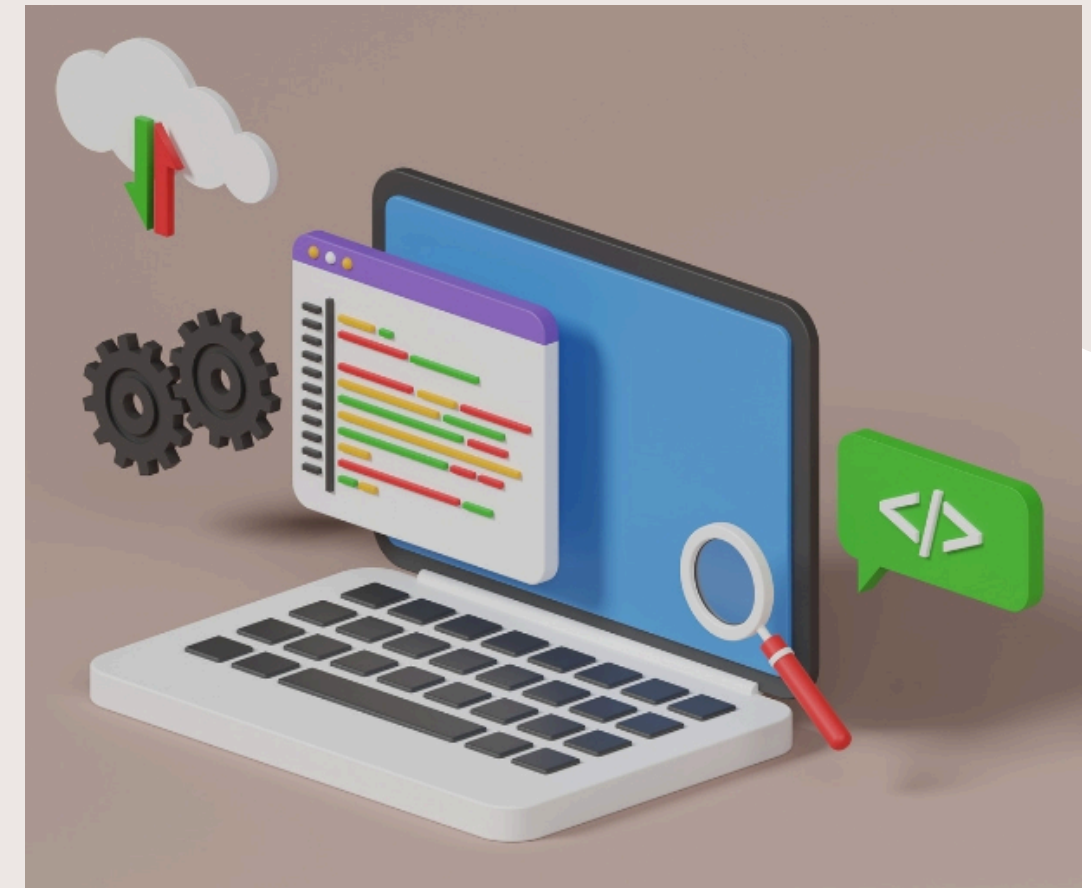
INTEGRANTES

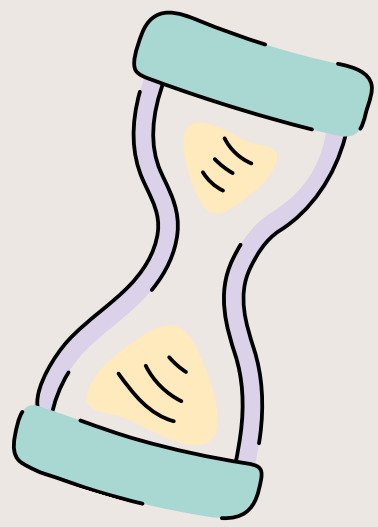
- RODRIGUEZ JAVIER, Yaziel Axel
- MOSTORINO HEREDIA, Leandro Enmanuel
- NESTARES ANDRADE, Fabricio Jesus
- ROBLES DAVILA, Bertrand Marcelo
- TOVAR PINZON, Nils Alexander



¿Qué es la POO?

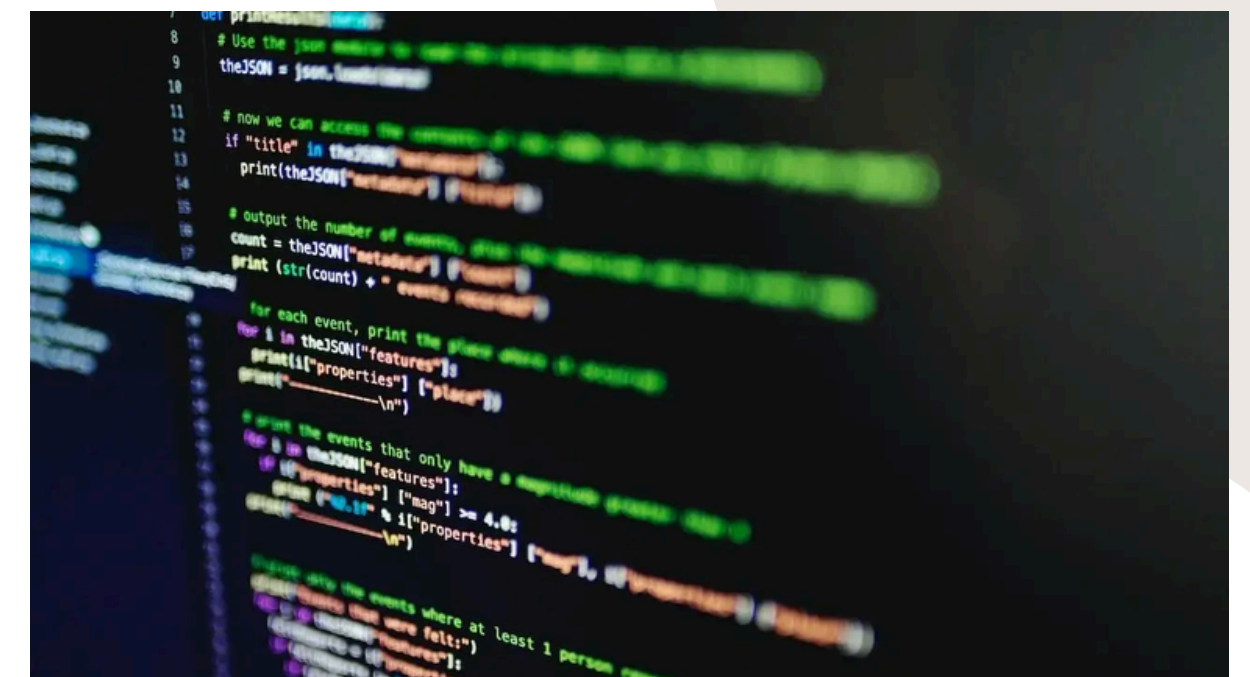
- Paradigma de programación basado en objetos.
- Agrupa datos (atributos) y acciones (métodos).
- Facilita organización, reutilización y mantenimiento del código.



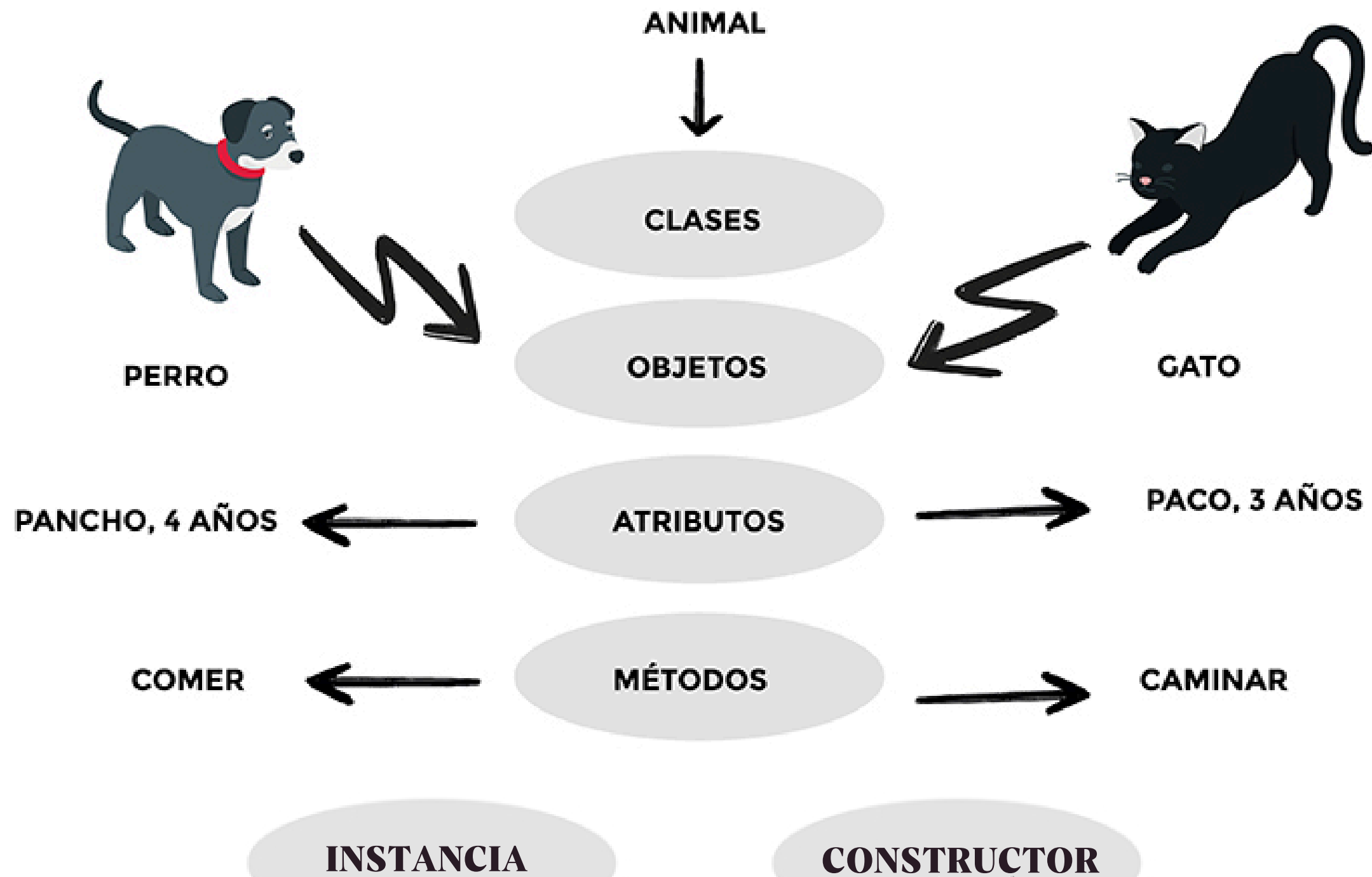
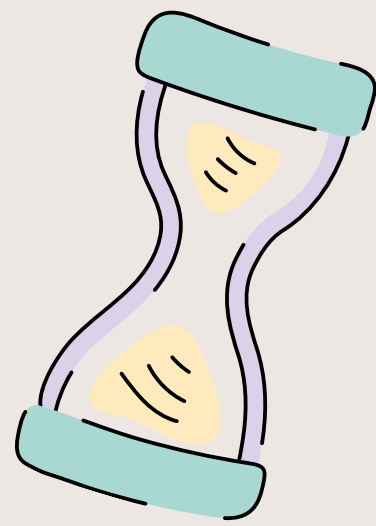


Ventajas de la P00

- Orden y estructura
- Reutilización de código
- Facilita mantenimiento
- Permite crear modelos del mundo real

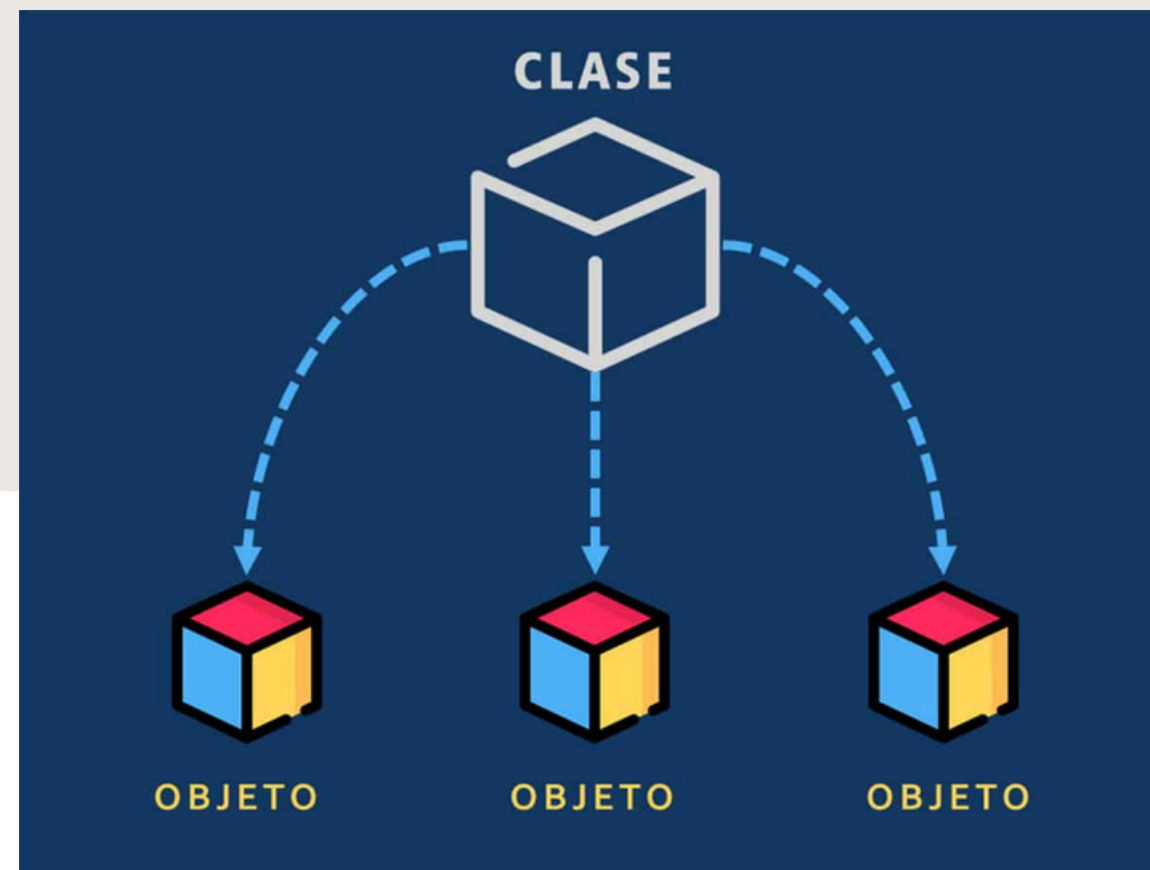


CONCEPTOS DE POO



¿QUÉ ES UNA CLASE?

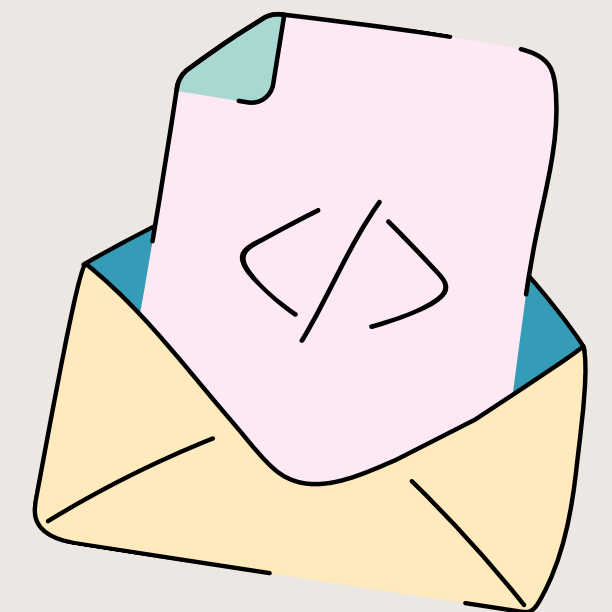
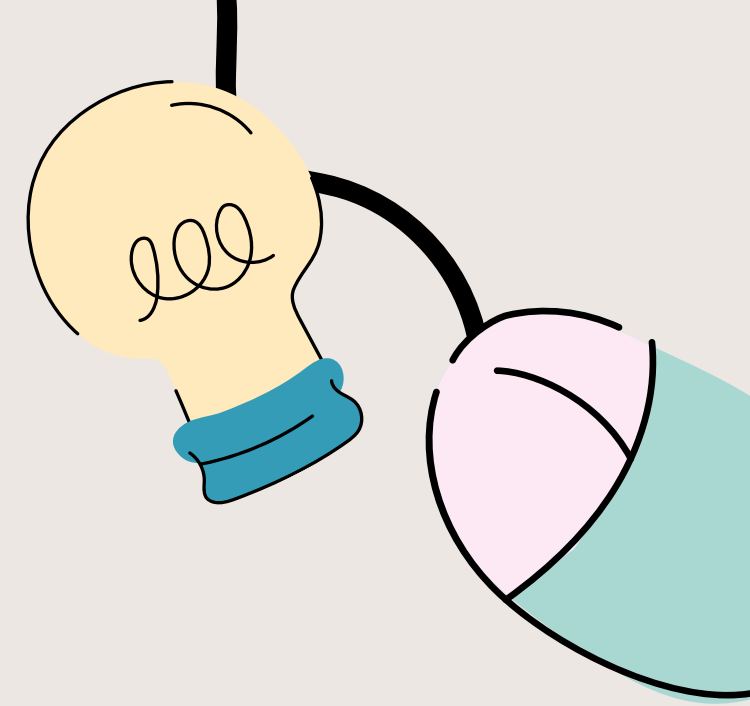
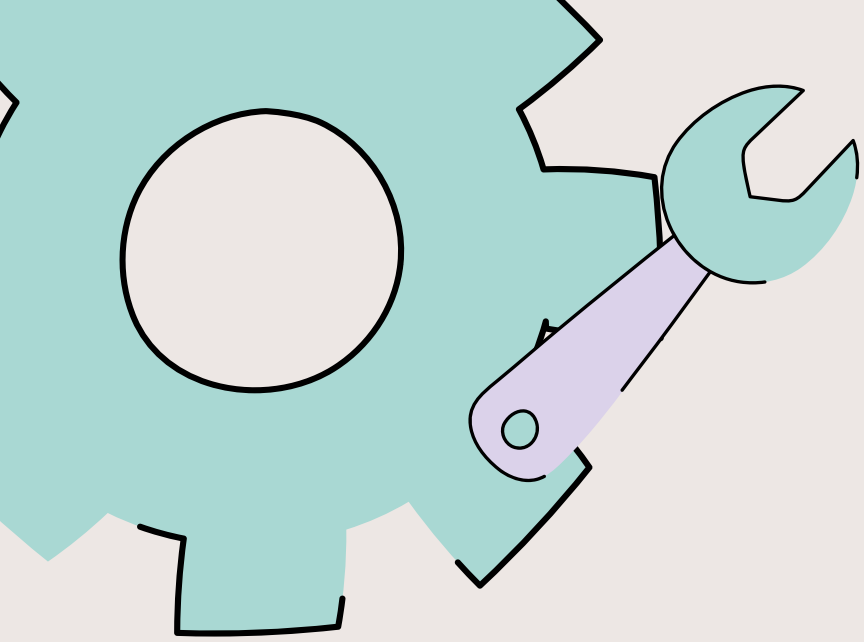
- Plantilla para crear objetos
- Define atributos y métodos
- Usa constructor()



Objetos (Instancias)

- Es una instancia de una clase
- Tiene atributos (datos)
- Tiene métodos (acciones)
- Representa algo del mundo real

Constructores, Getters y Setters en JavaScript



¿Qué es un constructor en JavaScript?

- Es un método especial dentro de una clase.
- Sirve para crear e inicializar objetos.
- Se ejecuta automáticamente cuando se usa **new**.

Ejemplo:

```
class Persona {  
  constructor(nombre, edad) {  
    this._nombre = nombre;  
    this._edad = edad;  
  }  
}
```


2. ¿Qué son los Getters?

- Métodos que permiten obtener el valor de una propiedad.
- Protegen los datos evitando el acceso directo.

Ejemplo:

```
get nombre() {  
    return this._nombre;  
}
```

3. ¿Qué son los Setters?

- Métodos que permiten cambiar una propiedad de forma controlada.
- Útiles para validar datos antes de asignarlos.

Ejemplo:

```
set nombre(nuevoNombre) {  
    this._nombre = nuevoNombre;  
}
```

4. ¿Por qué Usarlos Juntos?

- ✓ Encapsulación

Evitan modificar datos directamente.

- ✓ Validación de datos

Permiten revisar que los valores sean correctos.

- ✓ Código más limpio y profesional

Acceso natural:

```
persona.nombre = "Luis";  
console.log(persona.nombre);
```

5. Ejemplo Completo

```
class Producto {
  constructor(nombre, precio) {
    this._nombre = nombre;
    this._precio = precio;
  }

  get nombre() {
    return this._nombre;
  }

  set nombre(nuevoNombre) {
    this._nombre = nuevoNombre.trim();
  }

  get precio() {
    return this._precio;
  }

  set precio(nuevoPrecio) {
    if (nuevoPrecio > 0) {
      this._precio = nuevoPrecio;
    } else {
      console.log("Precio inválido.");
    }
  }
}
```

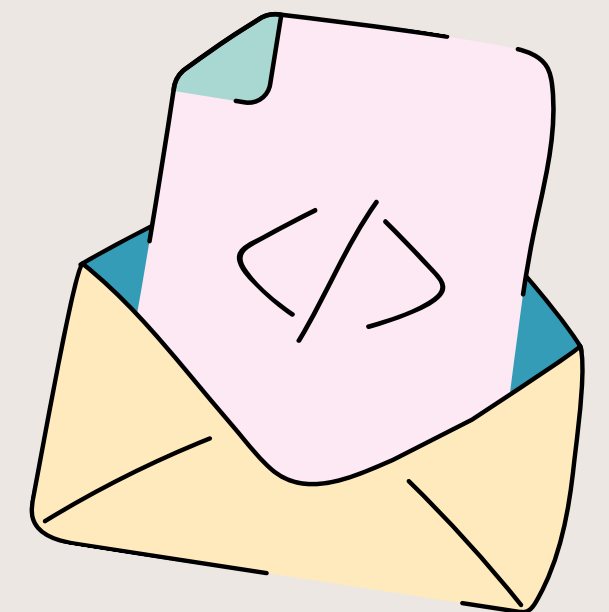
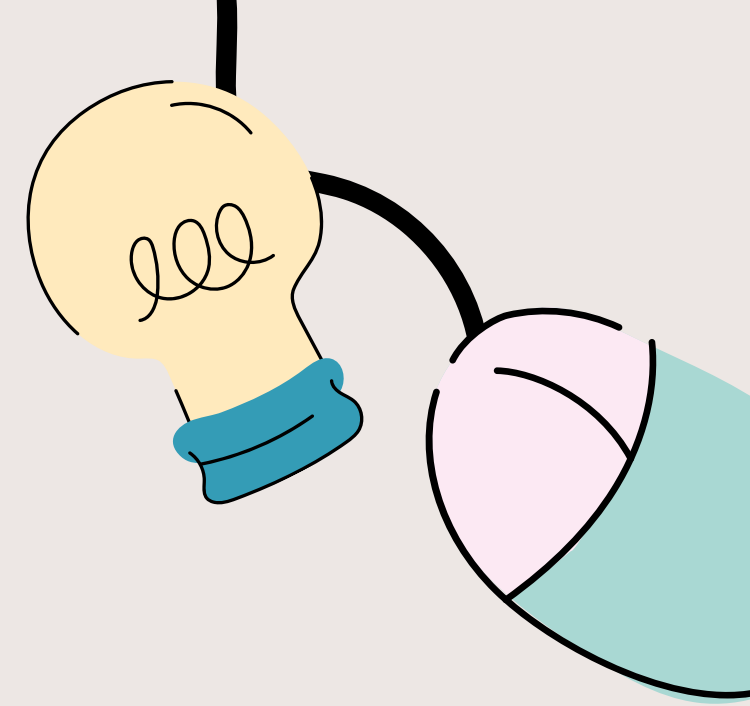
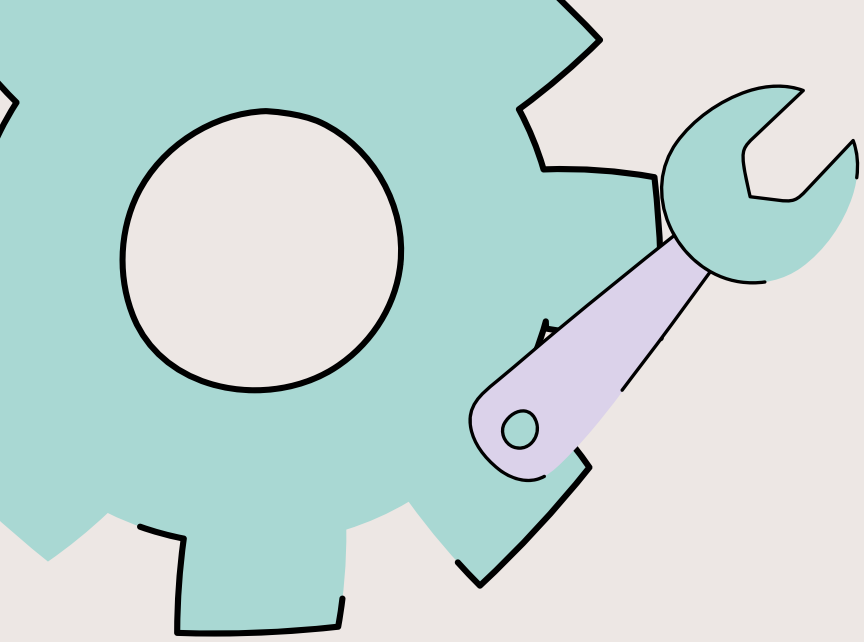
6. Beneficios en Proyectos (Visual Studio Code)

- Organización del código usando POO.
- Fácil mantenimiento.
- Reutilización de clases.
- Manejo seguro de datos.

7. Conclusión

Los constructores con getters y setters permiten crear clases limpias, seguras y profesionales, ideales para proyectos de JavaScript en Visual Studio Code.

Metodos Estáticos y Clase Math



¿Qué significa la palabra clave **STATIC**?

La palabra clave **static** se aplica a miembros de una clase (métodos o propiedades) y significa que ese miembro pertenece a la **clase en sí**, y no a una instancia específica de la clase (un objeto).

Pertenencia a la Clase

El método o propiedad existe tan pronto como la clase es cargada en memoria, independientemente de cuántos objetos se creen.

Independencia de la Instancia

No depende del estado interno de ningún objeto creado a partir de esa clase. Es compartido por todas las instancias.

Utilidades y Funciones Generales de Métodos Estáticos

Los métodos estáticos se utilizan para funciones que son **utilidades generales** o que operan en datos de entrada sin modificar el estado de un objeto.

1

Funciones de Utilidad

Creación de funciones auxiliares que no necesitan datos de instancia.

Ejemplo: formateo de fechas, validación de datos, conversión de unidades.

2

Métodos de Fábrica

Creación de objetos de la propia clase o de otras clases.

Ejemplo: patrones de diseño Factory para instanciación controlada.

3

Contadores de Instancias

Mantener un registro del número de objetos creados.

Ejemplo: contar cuántas instancias de una clase existen en memoria.

La clase MATH: Una utilidad fundamental

La clase **Math** es una **clase ya creada** (nativa en lenguajes como JavaScript o Java) que proporciona propiedades y métodos para constantes y funciones matemáticas comunes.

✓ **Propósito:**
Realizar operaciones matemáticas sin la necesidad de escribir la lógica desde cero

✓ **Más Constantes:** **Math.E** (la base del logaritmo natural)

✓ **Constantes:** **Math.PI** (el valor de Pi)

✓ **Ventaja:** Disponible sin necesidad de instanciación de objetos

Todos los métodos de MATH son estáticos

Una característica clave de la clase **Math** es que **todos sus métodos son estáticos**. Esto se debe a que las operaciones matemáticas no dependen del estado de un objeto específico.

→ Diseño de Utilidad:

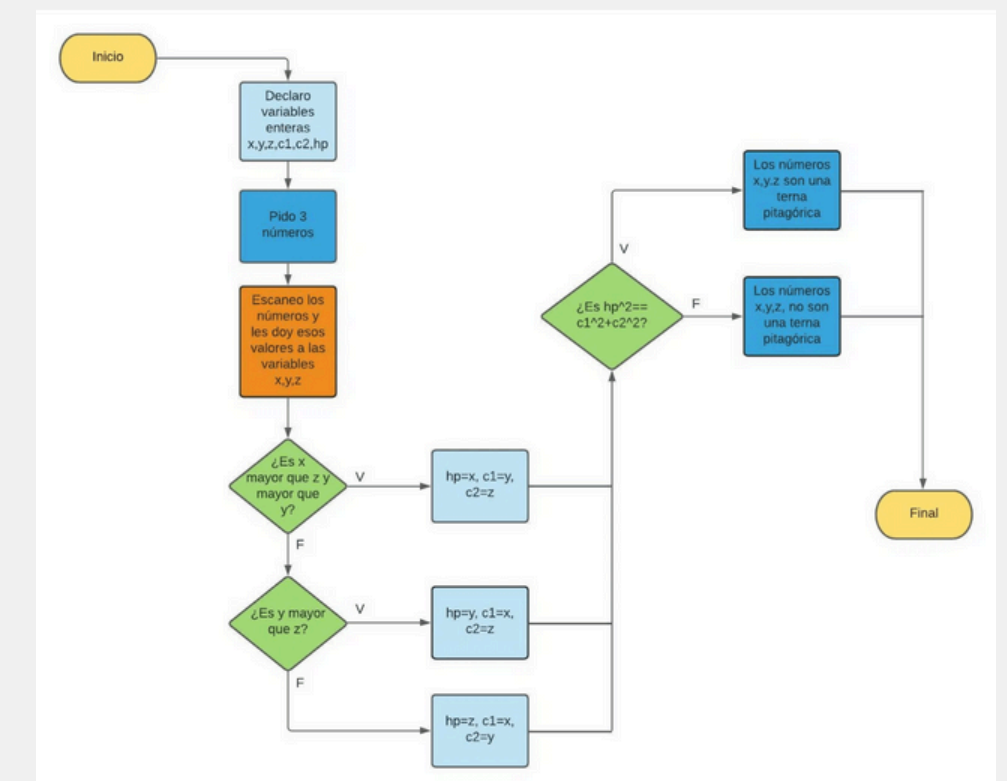
Las operaciones matemáticas (como calcular una raíz cuadrada o un seno) no dependen del estado de un objeto Math.

→ Invocación Simple:

Se accede a todas las funciones directamente a través del nombre de la clase, sin instanciación.

→ Ejemplo: **Math.cos(0)** funciona sin crear un objeto Math.

→ Eficiencia: Evita la sobrecarga de crear múltiples instancias innecesarias.



Métodos Comunes de la Clase Math

La clase **Math** ofrece una amplia gama de funciones útiles para el desarrollo web. Los tres métodos más comunes son:

Math.random()

Genera un número pseudoaleatorio entre 0 (incluido) y 1 (excluido).

```
Math.random() // Resultado: 0.4567...
```

Math.sqrt(x)

Devuelve la raíz cuadrada de un número x.

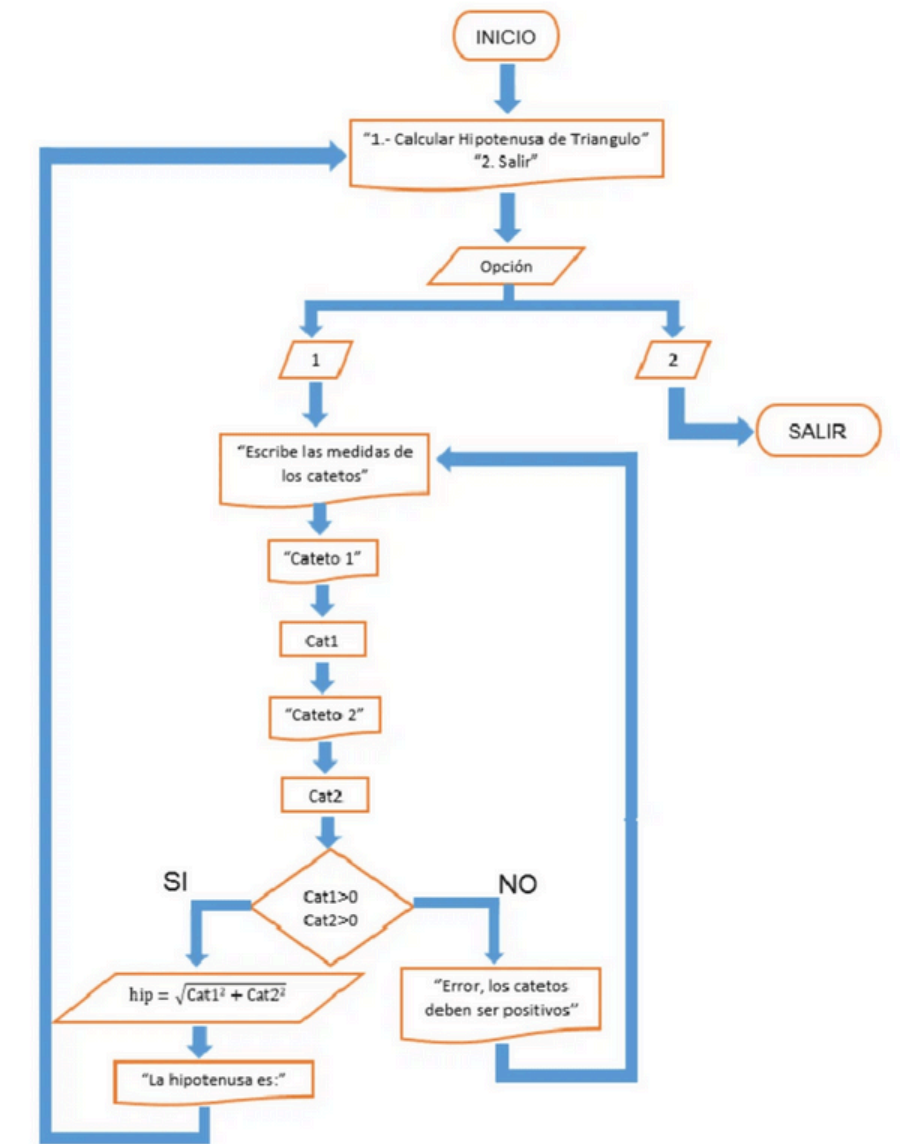
```
Math.sqrt(16) // Resultado: 4
```

Math.pow(base, exponente)

Devuelve la base elevada a la potencia del exponente.

```
Math.pow(2, 3) // Resultado: 8
```

DIAGRAMA DE FLUJO



Conclusión: Métodos Estáticos y la Clase Math

1

Funciones de Utilidad

Los métodos estáticos son funciones que no requieren instanciación de objetos.

2

Acceso Directo

Se invocan directamente a través del nombre de la clase sin usar **new**.

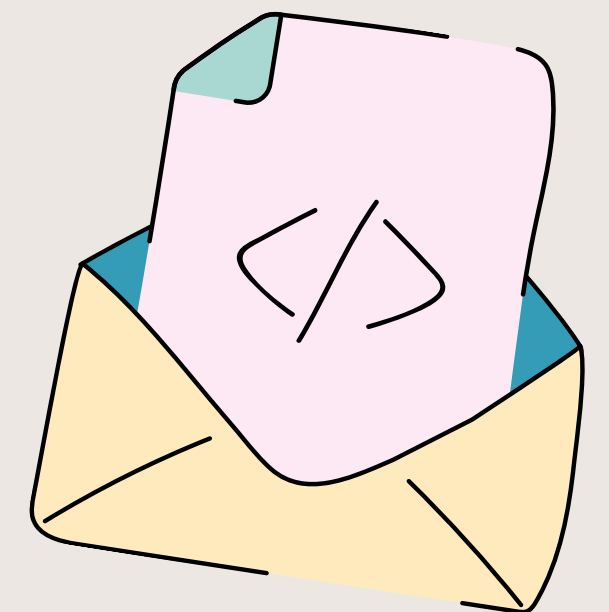
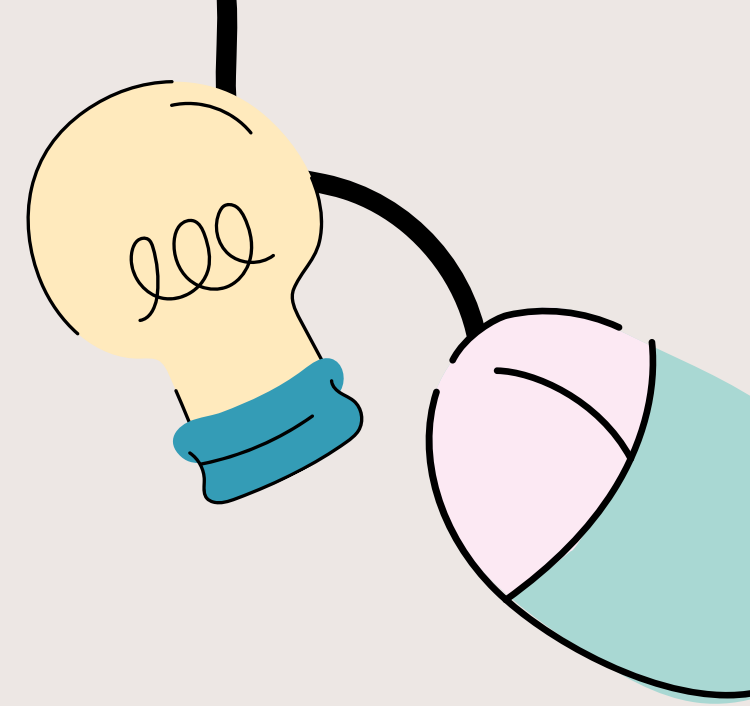
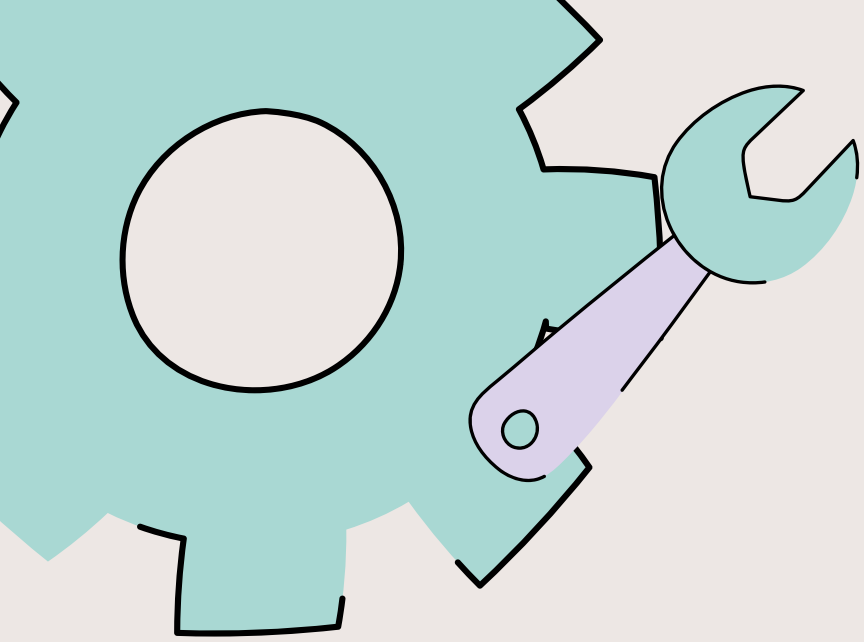
3

Ejemplo Canónico

La clase **Math** es el ejemplo perfecto de utilidad con todos sus métodos estáticos.

Los métodos estáticos son una herramienta **fundamental en el desarrollo web dinámico**, permitiendo crear código más eficiente, reutilizable y fácil de mantener.

Herencia en JavaScript



¿Qué es la herencia en JavaScript?

La herencia es un mecanismo que permite que una clase (o función constructora) adquiera propiedades y métodos de otra. Su finalidad es reutilizar código, modelar jerarquías lógicas y permitir extender funcionalidades sin duplicación.

Sirve para reutilizar código y crear objetos más especializados sin volver a escribir todo desde cero.

En JavaScript existen dos niveles de herencia:

- Herencia basada en prototipos (modelo nativo del lenguaje)
- Herencia basada en clases (class y extends) introducida en ES6, pero implementada sobre el sistema de prototipos..

Ejemplo en palabras:

- Animal: tiene nombre y hace sonidos.
- Perro: es un Animal, pero con un sonido especial.

Perro hereda todo lo de Animal.

¿Cómo se usa la herencia con extends?

JavaScript usa la palabra extends para indicar que una clase viene de otra:

```
class Hija extends Padre { }
```

Significa:

- "Hija" tiene todo lo que tiene "Padre".
- "Hija" puede agregar más cosas.
- "Hija" puede cambiar lo que quiera.

¿Qué es super()?

Cuando creas una clase hija, debes llamar a super() dentro del constructor.

¿Por qué?

Porque super() ejecuta el constructor del padre y activa todo lo que la clase padre necesita:

```
class Animal {  
    constructor(nombre) {  
        this.nombre = nombre;  
    }  
}  
  
class Perro extends Animal {  
    constructor(nombre, raza) {  
        super(nombre); // Llama al constructor de Animal  
        this.raza = raza;  
    }  
}
```

Heredar métodos

Si la clase padre tiene métodos, la clase hija los obtiene automáticamente:

```
class Animal {  
  hacerSonido() {  
    console.log("Sonido genérico");  
  }  
}  
  
class Perro extends Animal {}  
  
const p = new Perro();  
p.hacerSonido(); // Funciona porque lo heredó del padre
```


Sobrescribir métodos

Una clase hija puede cambiar un método heredado:

```
class Animal {  
  hacerSonido() {  
    console.log("Sonido genérico");  
  }  
}  
  
class Perro extends Animal {  
  hacerSonido() {  
    console.log("Guau!");  
  }  
}
```

Incluso puede usar el del padre y modificarlo:

```
hacerSonido() {  
  super.hacerSonido(); // Llama al método del padre  
  console.log("Pero más fuerte: GUAU!");  
}
```

¿Para que sirve la herencia en JavaScript?

Usa la herencia cuando:

- Un objeto **es una versión más especializada** de otro.
- Ejemplos:
 - Perro es un Animal.
 - Camión es un Vehículo.
 - Usuario_Admin es un Usuario.

La herencia hace tu código:

- Más ordenado
- Más corto
- Más fácil de mantener

OBJETO DATE

El objeto Date en JavaScript permite trabajar con fechas y horas. Con él puedes obtener el día actual, sumar fechas, formatear horas, etc.

```
C: > Users > W10 > Downloads > JS tarea.js > ...  
1  let hoy = new Date();  
2  let fechaEspecifica = new Date(2025, 0, 15);  
3  let fechaCompleta = new Date("2025-12-10");
```

en este código de java script estamos creando fechas

a continuacion obtenemos componentes de la fecha

C: > Users > W10 > Downloads > JS tarea.js

```
1 hoy.getFullYear();  
2 hoy.getMonth();  
3 hoy.getDate();  
4 hoy.getDay();  
5 hoy.getHours();  
6 hoy.getMinutes();  
7 hoy.getSeconds();  
8
```

modificamos componentes

```
hoy.setFullYear(2030);  
hoy.setMonth(5);  
hoy.setDate(20);
```

formatos comunes :

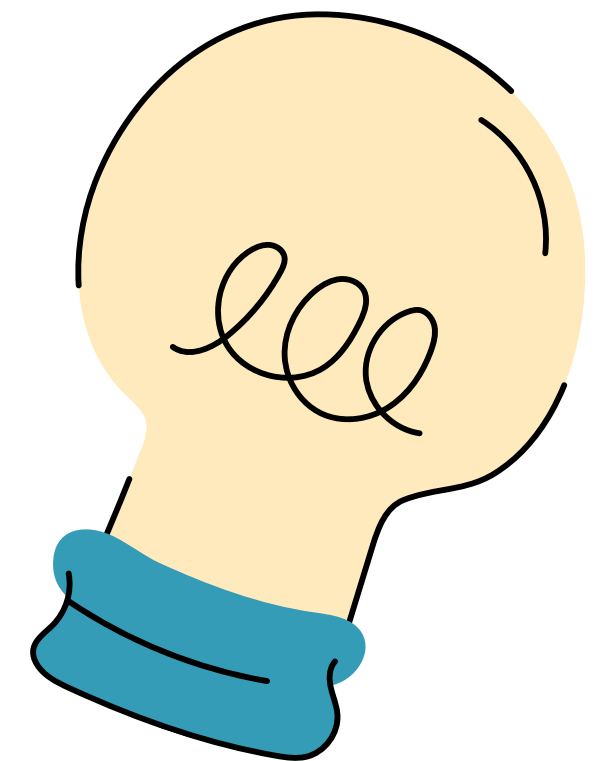
```
14
15 hoy.toString(); // "Mon Dec 10 2025"
16 hoy.toLocaleDateString(); // Fecha formateada según país
17 hoy.toISOString(); // Formato estándar ISO
18 |
```

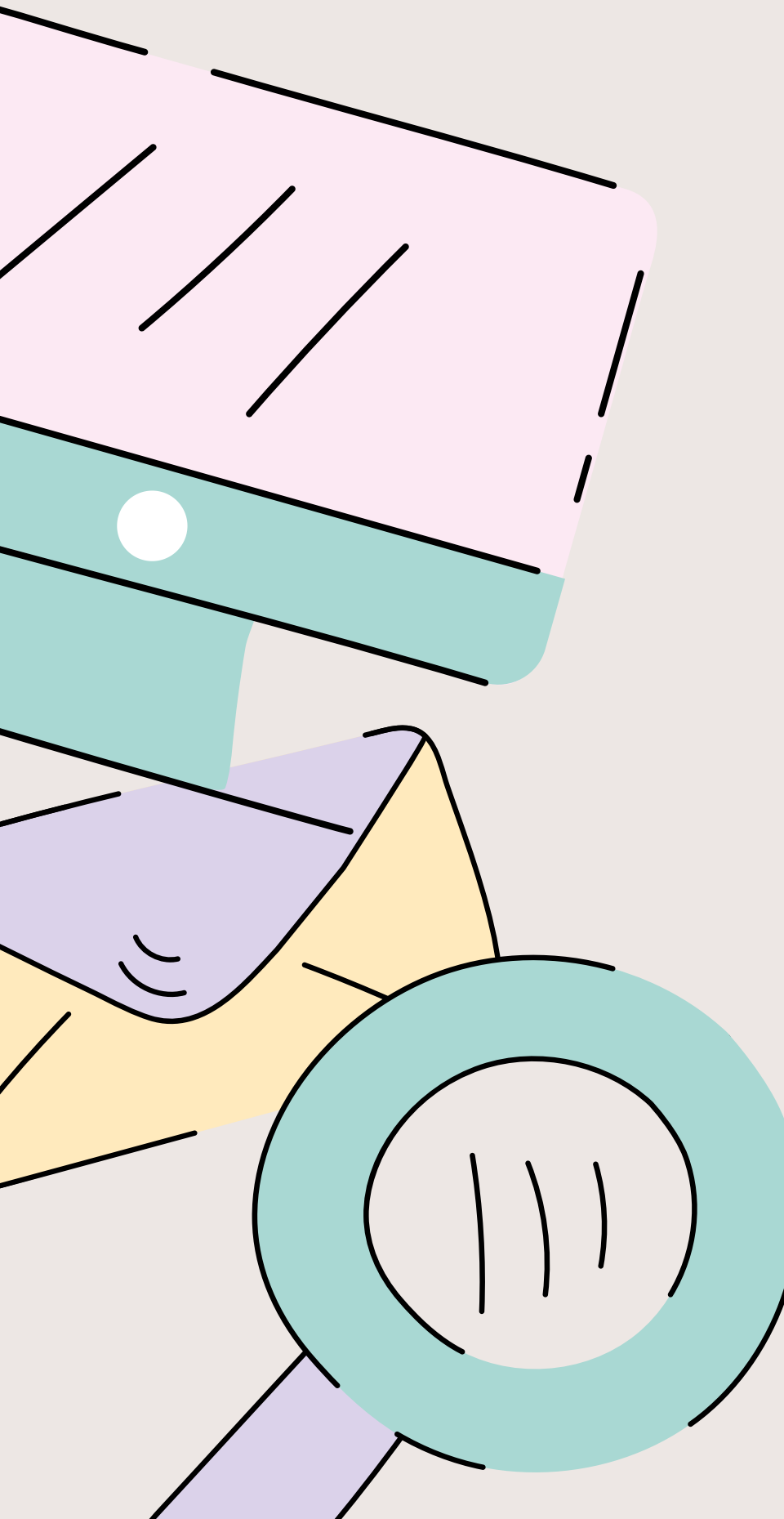
ejemplo práctico:

```
1 hoy.getFullYear();
2 hoy.getMonth();
3 hoy.getDate();
4 hoy.getDay();
5 hoy.getHours();
6 hoy.getMinutes();
7 hoy.getSeconds();
8
9
10 hoy.setFullYear(2030);
11 hoy.setMonth(5);
12 hoy.setDate(20);
13
14
15 hoy.toString(); // "Mon Dec 10 2025"
16 hoy.toLocaleDateString(); // Fecha formateada según país
17 hoy.toISOString(); // Formato estándar ISO
18
19
20 let fecha = new Date();
21 console.log("Hoy es:", fecha.toLocaleDateString());
22 console.log("Hora actual:", fecha.toLocaleTimeString());
23 |
```


CONCLUSIONES GENERALES DE POO

- La Programación Orientada a Objetos es un paradigma fundamental que permite desarrollar software de manera más organizada, modular y cercana a la realidad. Al basarse en clases y objetos, facilita representar elementos del mundo real mediante atributos y comportamientos, lo que mejora la comprensión del código y su mantenimiento. Además, sus principios —como la abstracción, el encapsulamiento, la herencia y el polimorfismo— permiten crear sistemas más seguros, flexibles y reutilizables, donde cada componente cumple un rol claro y puede evolucionar sin afectar al resto del programa. En conjunto, la POO se convierte en una herramienta esencial para construir aplicaciones escalables y de calidad, y sigue siendo uno de los enfoques más utilizados y valorados en el desarrollo de software moderno.





Gracias



hola@unsitiogenial.es

